

Navigating Architecture Decisions in Team Projects

A Practical Toolkit for Capstone Teams

Quick Question

How many of you have already had your first architecture disagreement?

- Someone wants microservices. Someone wants a simple monolith.
- One person picked a framework before the team could discuss it.
- You spent 45 minutes talking about a database solution and got nowhere.
- **If any of this sounds familiar you are exactly where you should be.**



Why Architecture Decisions Matter Right Now

Cost of Change

Architectural changes get exponentially more expensive the longer you wait. What takes an hour to rethink today could take a sprint to unwind in week 8.

2 Weeks In = Sweet Spot

You have enough context to make real decisions but not so much built code that bad choices are baked in. Act now.

Team Alignment

Unspoken assumptions are the #1 source of rework. Naming your architecture decisions out loud creates shared understanding.



What Counts as an Architecture Decision?

Structural choices that constrain how the rest of your system is built.

Architecture Decisions

- Monolith vs. microservices
- REST vs. GraphQL vs. gRPC
- SQL vs. NoSQL database choice
- Frontend framework selection
- Authentication strategy
- Deployment model (cloud, serverless, containerized)

Implementation Details (not architecture)

- Variable naming conventions
- Which CSS utility library to use
- Code formatting / linting rules
- Which team member owns which file



Common Traps Teams Fall Into

Over-Engineering Early

Designing for 10M users in week 2 of a capstone project. Solve the problem you have, not the one you imagine.

Unspoken Assumptions

Everyone thinks they agreed but each person built a different mental model. Verbalize and document decisions.

Loudest Voice Wins

Technical decisions made by whoever talks most, not whoever has the best reasoning. Use structured frameworks to equalize.

Copy Without Context

"Netflix uses microservices" is not a reason. Understand the tradeoffs before you adopt a pattern.



Start Here Before Any Framework

"What problem are we actually solving?"

- Every architecture debate should return to this question.
- Frameworks are tools they help you decide, not decide for you.
- Skipping this step is why teams end up debating the wrong thing for an hour.
- **Write the problem statement down before you open a whiteboard.**

The Frameworks Toolkit

Tools to help your team decide together

Tool 1: Architecture Decision Records (ADRs)

Best for: Documenting committed decisions

What it is

A short-written record of a decision your team made capturing the context, the options considered, and the tradeoffs accepted.

Tip: Even one ADR per major decision is better than none. It forces the team to articulate the why.

ADR Template

Title: ADR-001 [Short decision name]

Status: Proposed / Accepted / Deprecated

Context: What situation drove this decision?

Decision: What did we choose and why?

Consequences: What are the tradeoffs we accept?



Tool 2: Decision Matrix

Best for: Comparing multiple concrete options

What it is

Score your options against criteria that matter to your project. Weight each criterion by importance, then multiply and total.

Tip: The act of choosing your criteria as a team is often more valuable than the final score.

Criteria (Weight)	Option A	Option B	Option C
Performance (3×)	3 → 9	4 → 12	2 → 6
Team Familiarity (2×)	4 → 8	2 → 4	5 → 10
Simplicity (2×)	5 → 10	3 → 6	4 → 8
Total Score	27	22	24



Tool 3: Pro / Con / Tradeoff Table

Best for: Quick alignment when options are narrowed down

Faster than a full matrix great for time-boxed team discussions (set a 15-min timer).

Pros

- Simple to implement
- Team already knows it
- Well documented

Cons

- Doesn't scale past MVP
- Harder to test in isolation
- Vendor lock-in risk

Tradeoffs

- Fast now, refactor later
- Good enough for a 15-week project
- Revisit at Week 8

Tool 4: RACI / "Who Decides?" Map

Best for: Resolving ownership confusion and team dynamics

Use when your team is stuck not on the *what* but the *who*. Clarifies who has final say on a decision.

Responsible

Does the work / makes the call

Accountable

Owens the outcome, final sign-off

Consulted

Gives input before the decision

Informed

Notified after the decision is made

Tip: For capstone teams, one person should be Accountable per major decision. Shared accountability = no accountability.

Tool 5: The "Two-Way Door" Test

Best for: Right-sizing how much energy to spend on a decision

Not all decisions deserve the same amount of deliberation. Ask: can we walk this back?

Two-Way Door ↔

Reversible decision

Decide quickly, with enough discussion.
You can change it later without disaster.

Examples: API endpoint naming, folder structure, a library choice early in the project.

One-Way Door →

Irreversible (or costly to reverse)

Slow down. Use a deeper framework.
Get the team aligned before committing.

Examples: Core database type, cloud provider, authentication architecture.



Choosing the Right Tool for the Moment

Mix and match consistency matters more than picking the "right" one.

If your team is...	Try this tool
Committing to a decision and need to document it	ADR
Comparing 3+ concrete options objectively	Decision Matrix
Narrowed to 2 options and need quick alignment	Pro / Con / Tradeoff
Stuck because no one knows who decides	RACI Map
Spending too much time on a minor decision	Two-Way Door Test
Totally paralyzed? Try any of the above!	Start with the Core Question



Communicating Your Architecture

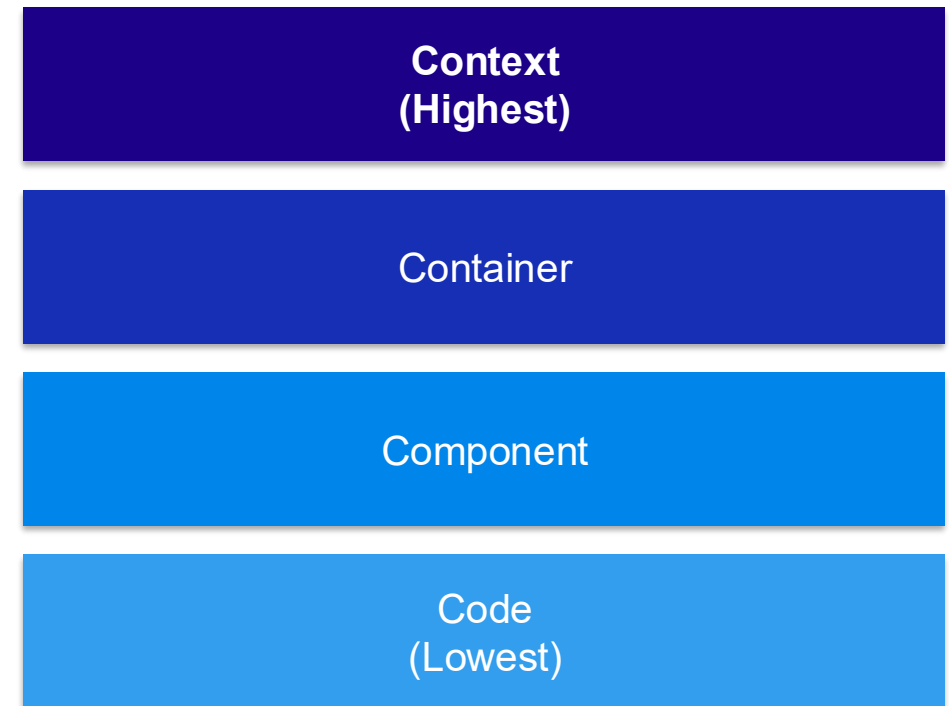
Using the C4 Model to draw at the right level of detail

Why Diagrams Matter

A decision made but not visualized is a decision not shared.

- Diagrams reduce ambiguity across roles: dev, PM, stakeholder, QA
- Different audiences need different levels of detail ("resolution")
- A diagram is a conversation starter, not a deliverable
- The act of drawing forces you to surface hidden assumptions
- Outdated diagrams are worse than none, keep them lightweight

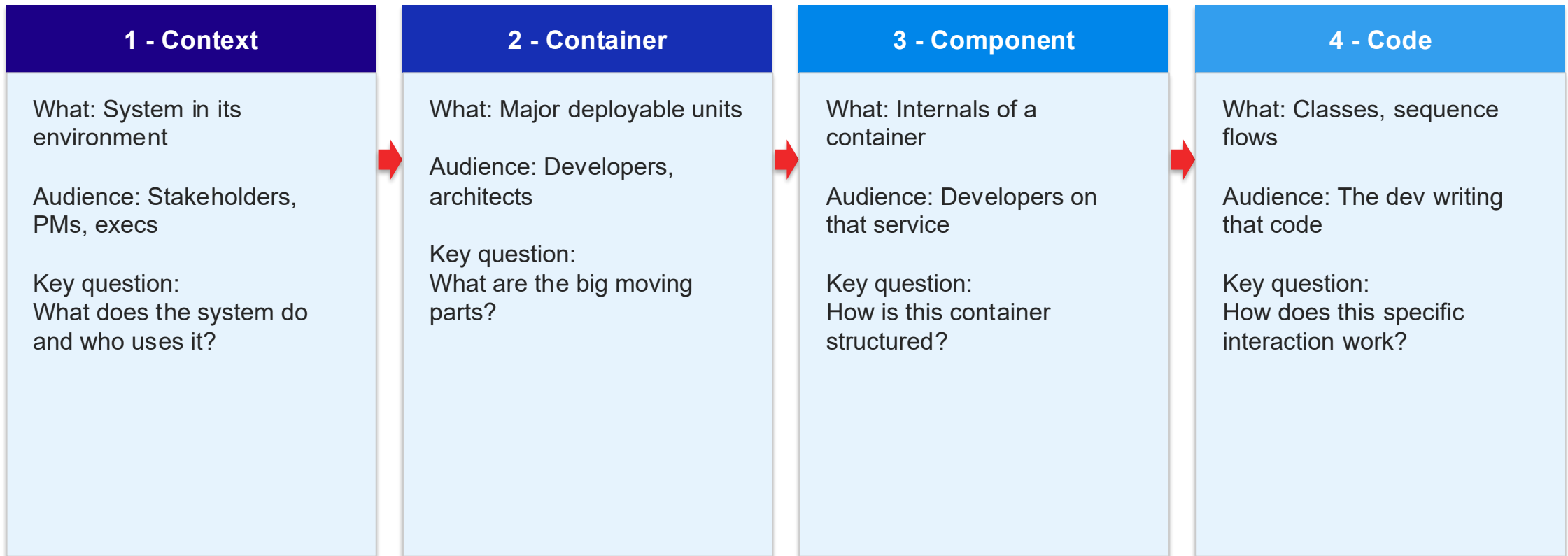
Resolution Spectrum



① Other frameworks: Arc42 uses 12 structured sections; the 4+1 View Model organizes diagrams around logical, process, physical, development, and scenario views. C4 is the most accessible starting point for student teams.

The C4 Model - Four Levels of Detail

Zoom in progressively and use the level your audience needs.



Level 1: Context Diagram

Audience: Stakeholders, PMs, non-technical reviewers

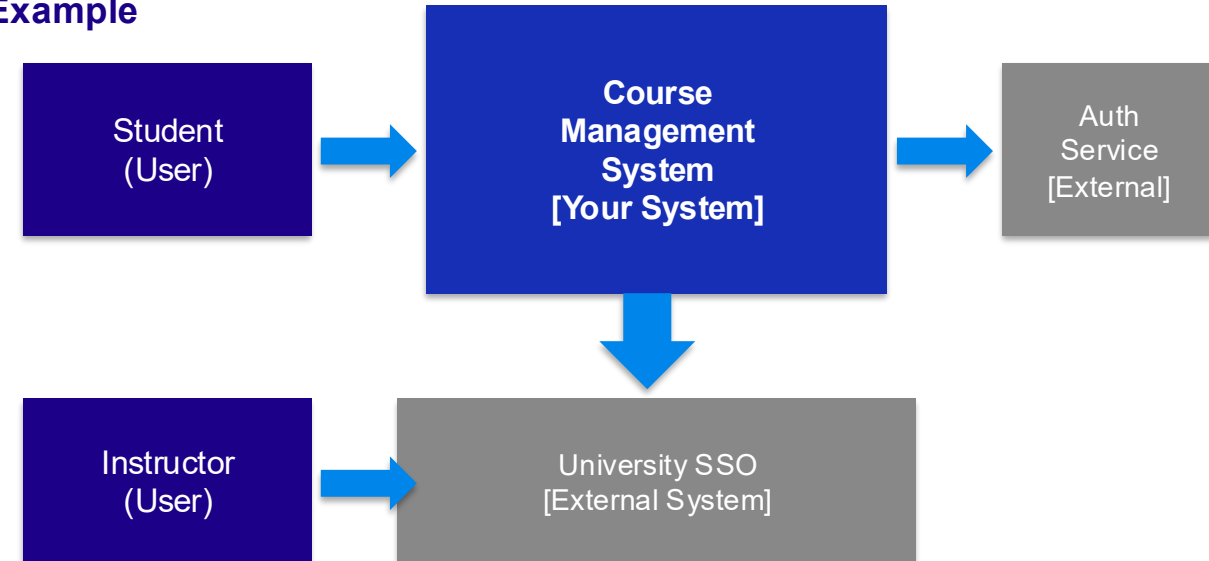
What it shows

- Your system as a single box "black box"
- External users and actors who interact with it
- External systems it connects to (APIs, services, databases outside your boundary)
- High-level relationships, no technology detail

When to use it

- Kickoff presentations, project proposals, stakeholder reviews

Example



No technology detail just who talks to what.

Level 2: Container Diagram

**Audience: Developers, architects,
technical leads**

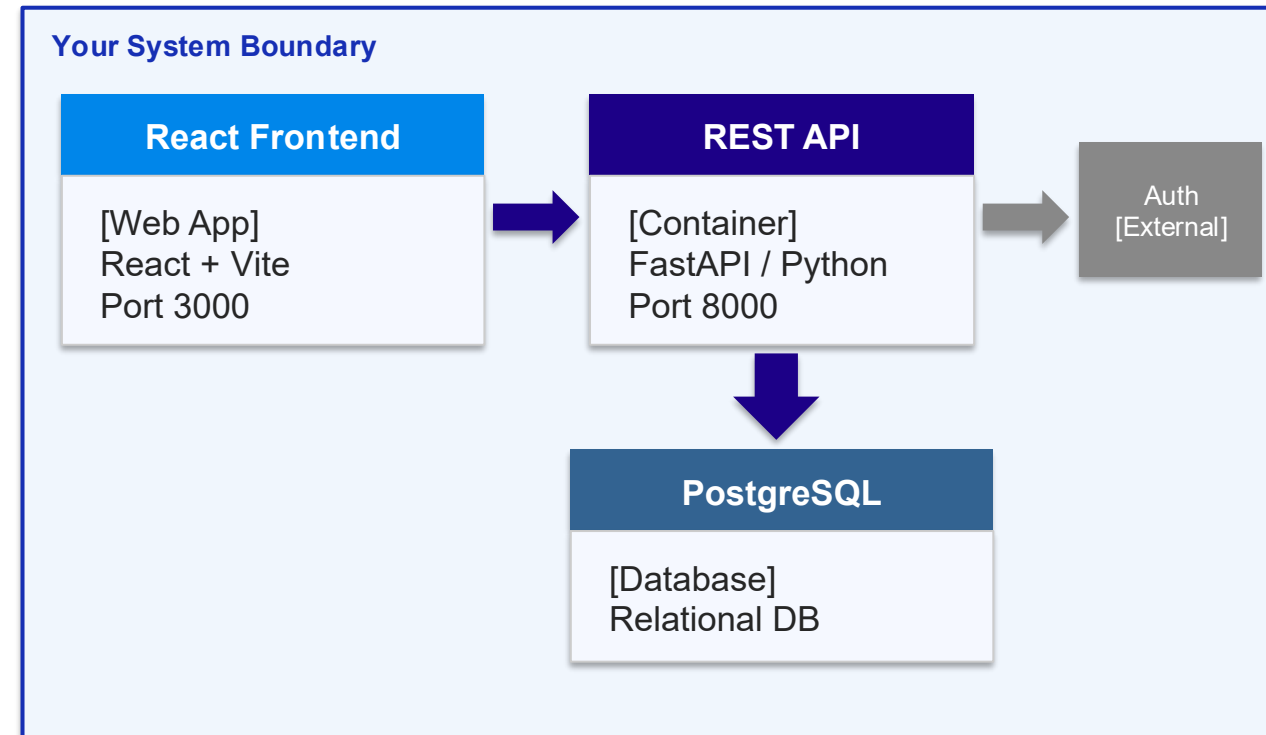
What it shows

- Major deployable units: web app, API, mobile app, database, message queue
- Technology choices at the container level (Node.js API, React frontend, PostgreSQL)
- How containers communicate (HTTP, WebSocket, message queue)

When to use it

- Sprint planning, architecture reviews, onboarding new team members

Example



Shows technology choices - the first place to surface big architectural decisions.



Level 3: Component Diagram

Audience: Developers working inside a specific container

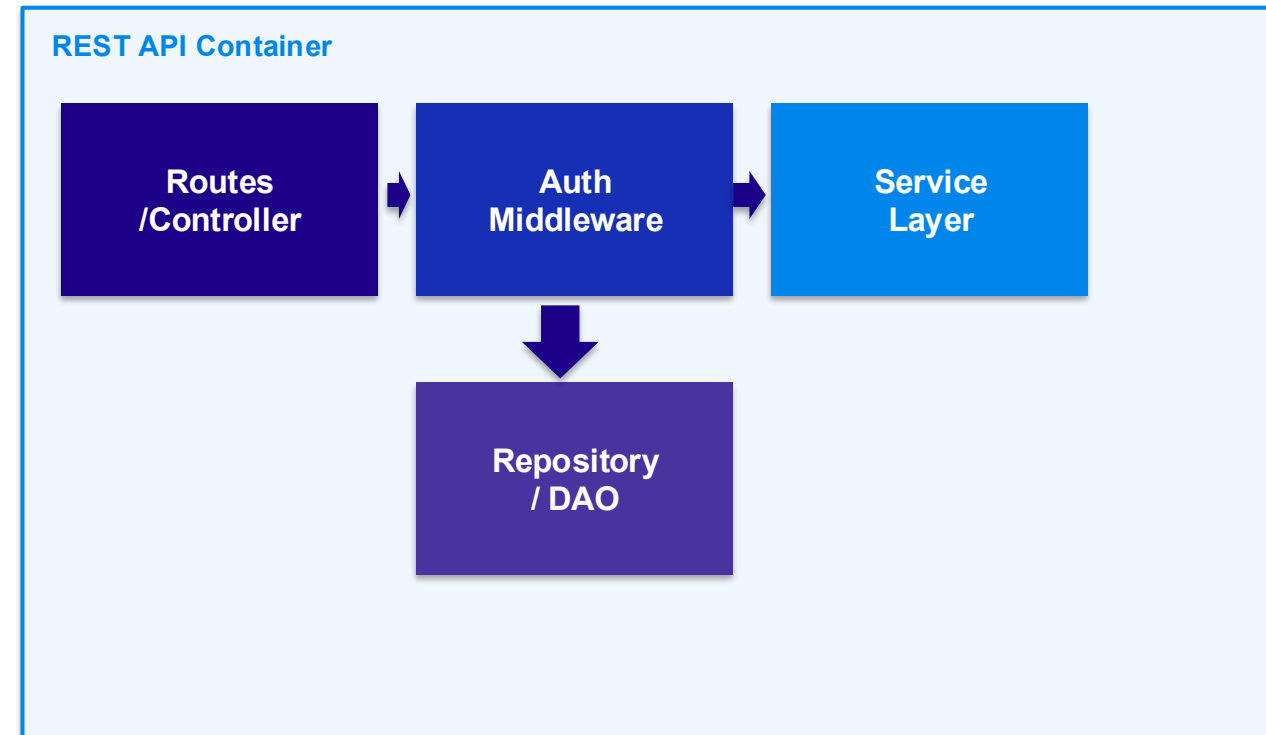
What it shows

- Internal structure of one container (e.g., your API)
- Components: controllers, services, repositories, middleware
- How those components depend on and talk to each other

When to use it

- When dividing work inside a container across team members

Example - Inside the REST API



Great for dividing work - each component can map to a team member.



Level 4: Code & Sequence Diagrams

Audience: The developer writing that specific interaction

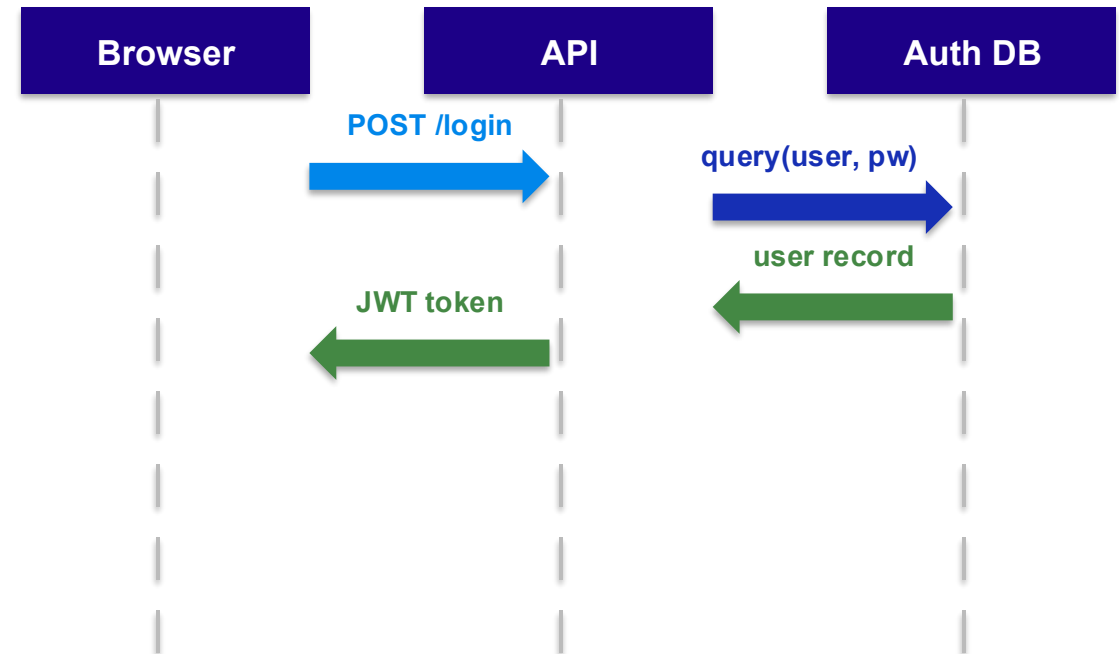
What it shows

- Sequence: the exact order of calls for a single user flow
- Class diagrams: object relationships and inheritance
- Optional at C4 - useful for complex or tricky flows

When to use it

- Debugging, code reviews, onboarding, documenting auth flows or payment flows

Example - Login Sequence



Use sequence diagrams for flows you will implement or review this sprint.



Choosing the Right Diagram

Match the diagram to your audience not to what you feel like drawing.

C4 Level	Draw it when...	Your audience is...	Tool suggestions
1 - Context	Starting the project or presenting to stakeholders	Non-technical / leadership	Excalidraw, draw.io, pen & paper
2 - Container	Planning architecture or onboarding new team members	Whole dev team	draw.io, Lucidchart, Mermaid
3 - Component	Dividing work inside a service across developers	Devs on that service	draw.io, PlantUML, Mermaid
4 - Code	Debugging, reviewing, or documenting a complex flow	The dev writing that code	PlantUML, Mermaid, VS Code ext.

Tip: Start every project with a Context diagram. It takes 15 minutes and saves hours of confusion.



Team Activity: Try It Now (10 min)

1

Step 1

Identify one current architecture question your team has not fully resolved.

2

Step 2

Draw a high-level container diagram that outlines internal/external actors and how they intend to collaborate with each other.

3

Step 3

Be ready to share: What assumptions did you surface with your diagrams?

 **Set a 20-minute timer**



Key Takeaways

1

Ask the core question first

"What problem are we actually solving?" Every architecture conversation starts here.

2

Name your decisions do not assume

Unspoken decisions become unresolved conflicts. Write it down, even briefly.

3

Right-size your effort

Use the Two-Way Door test to decide how much time a decision deserves.

This week: pick one framework and use it before your next standup.



Thank You!

Your team. Your decisions. Your architecture.

Make a new world happen